



Facial expression recognition

Our decisions, experiments and limits

Deep Learning · ING3 AI · Group B

Teacher: Hanane Zerdoum

Paul Pitiot and Maxime Crayssac

9 October 2026

Contents

Project overview	3
1 A dataset we could start using	4
2 Give each image one role	6
3 What our first network could learn	8
4 Why we tried convolutions	10
5 Which expressions were we missing?	13
6 Testing our explanations separately	14
7 What augmentation improved	15
8 A more careful search	18
9 Rejected settings still taught us something	21
10 Why S5 won, and what that means	22
11 Three photographs were not enough	25
12 What happens to one group photograph	29
Sources	33
Evidence index	34

Project overview

We set out to recognise seven facial expressions, first in small images of individual faces, then in photographs containing several people. We had one week to develop the project. That constraint influenced our choice of data, the size of our networks and the experiments we could finish.

FER2013 let us begin immediately with 35,887 cropped, grayscale images. We built a simple dense network as a reference, then a convolutional network that could learn local patterns in the image. Its validation accuracy rose from 36.08% to 55.65%, although the different training budgets prevent us from attributing the whole gain to architecture alone.

The first CNN also overfitted. We tried a smaller dense layer, Dropout and a lower learning rate separately, then added training-image augmentation. This produced A1, our first selected model. It reached 57.55% accuracy on the held-out test images. Its weak results on fear and disgust, together with small differences between some earlier experiments, made us want a more careful comparison.

We then trained 12 configurations across 28 runs. The six-step experiment ladder and the two most promising search candidates were compared over three training seeds. Our final classifier, S5, was selected using mean validation loss. It reached **62.44% test accuracy and 0.562 macro F1**. Its advantage over the simpler R4 candidate was small, and R4 had the better mean macro F1 on validation.

For whole photographs, we connected the classifier to a pretrained face detector. YuNet and YOLO11n-face tied on three Apollo crew photographs. A broader collection of 80 NASA photographs exposed differences: YOLO produced 42 additional boxes, 41 of which contained real faces on inspection. We accepted its extra processing time and made it the default detector.

Our final system uses **YOLO11n-face to locate faces and S5 to classify each crop**. Its labels describe visible expressions under our dataset conventions. They do not establish what a person feels. We consulted the same official test set twice, first for A1 and later for S5; both results remain recorded.

1 A dataset we could start using

We began on 1 October with a presentation scheduled for 9 October. We needed data that would let us spend most of that week learning from experiments. FER2013, in the Kaggle folder distribution maintained by msmbare, met that need. We could obtain it without waiting for an academic access request, and its seven categories matched the task we had chosen from the assignment: anger, disgust, fear, happiness, sadness, surprise and neutral.

The images were already cropped around faces. At 48×48 pixels in grayscale, each contained only 2,304 values. That made repeated training practical on Colab and later on our Mac. It also matched the first six assignment parts, which start with an extracted face. We could postpone localisation until we had a working expression classifier.

1.1 What we considered

Candidate	What mattered for our project
FER2013	Available immediately; cropped 48×48 faces; seven original labels. Selected.
RAF-DB	29,672 images overall, but 15,339 in the seven-class basic-expression subset. We were unsure how quickly we could obtain access; we did not measure a waiting time.
AffectNet	A larger collection. The original academic procedure involved a supervisor for student requests, adding coordination to our short schedule.
Aff-Wild2	Video and frame-level annotations, with an access agreement. We would also have needed to prepare a different workflow.
CAER / ExpW	Alternatives we considered without preparing experiments or fully checking access. We cannot describe them as unavailable.
FER+	New annotations for FER2013, with eight expressions plus unknown and non-face votes. A possible later improvement, never used in our runs.

Choosing FER2013 also meant accepting its limitations. It was collected from internet images and contains varied poses, imperfect crops and ambiguous labels. Grayscale removes colour cues, and 48×48 pixels leave little detail around the eyes and mouth. We encountered suspicious labels and a non-face image during inspection. Those observations later mattered when we interpreted confident mistakes.

Figure 1 shows five fitting images from each class. Even in this small sample, the pose, crop and lighting vary. Some expressions are hard to distinguish at this resolution, before a model makes any prediction.

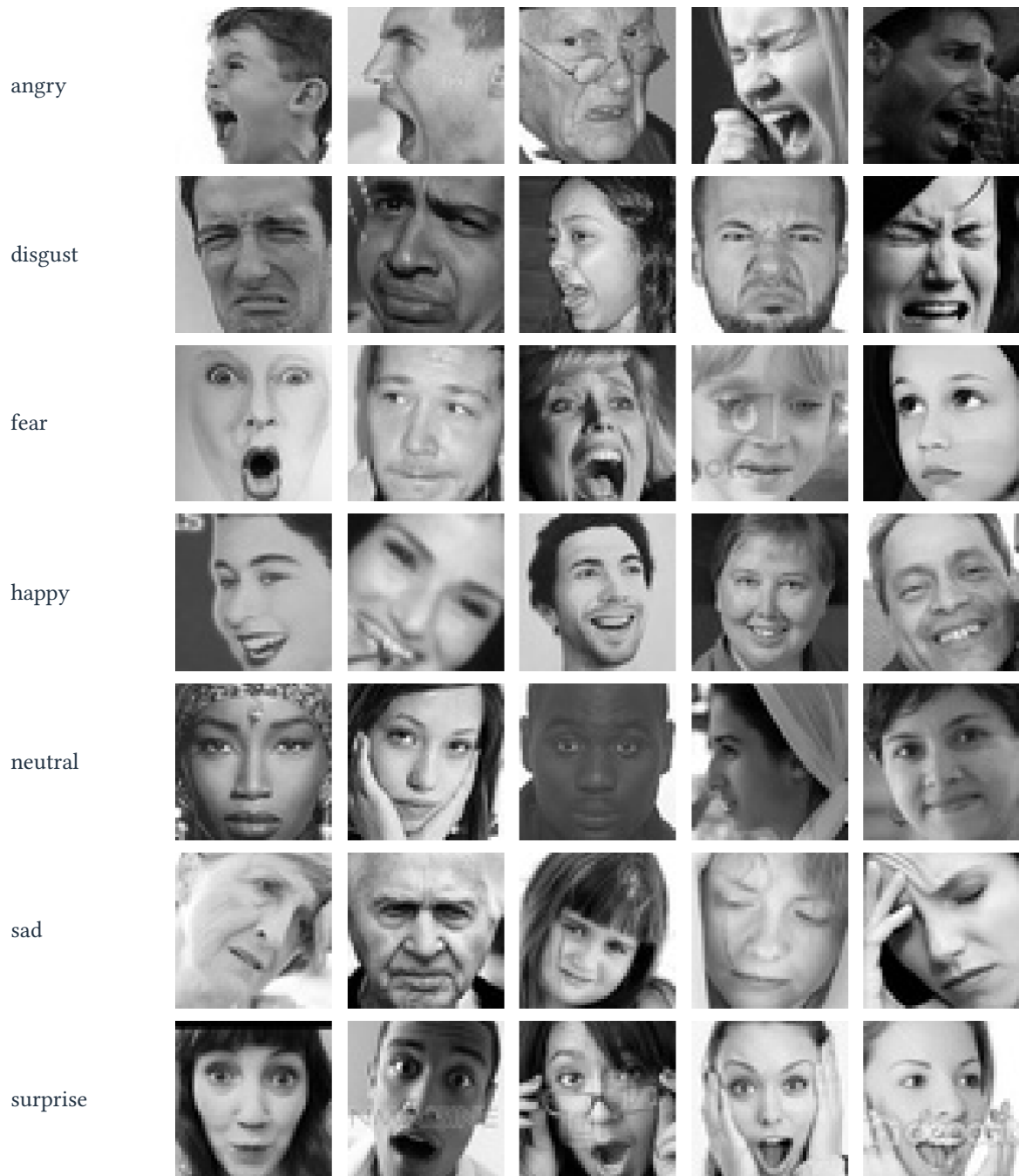


Figure 1: Five examples per original FER2013 class, taken from the fitting split. The labels are the dataset annotations, including their ambiguities. These images illustrate variation in the input; they are not verified accounts of how the people felt. Source: supplied notebook, class-sample output; P1 and P9.

Our choice was workable for the time available. It did not establish FER2013 as the best dataset for recognising expressions in every setting. We used its original single labels throughout; neither FER+ annotations nor a separately trained face detector entered the classification experiments.

Evidence: P1; R1, R2, R3, R5 to R10. Dataset characteristics and access procedures are external facts; the deadline and reasons for choosing are our recollections.

2 Give each image one role

The Kaggle distribution contains 28,709 training images and 7,178 test images. We created our validation set from the training folder: 24,402 images for fitting weights and 4,307 for comparing checkpoints and configurations. The split was stratified, preserving approximately the same class proportions, and fixed with seed 42. All subsequent experiments used the same split.

We kept the 7,178 test images out of weight fitting and configuration selection. Our original intention was one final evaluation. A1 was tested on 7 October; after reopening the investigation, S5 was tested on 8 October. Saving each evaluation once does not make the second result independent of knowledge gained from the first.

2.1 The imbalance was visible before training

Expression	Official train	Official test
Angry	3,995	958
Disgust	436	111
Fear	4,097	1,024
Happy	7,215	1,774
Neutral	4,965	1,233
Sad	4,830	1,247
Surprise	3,171	831
Total	28,709	7,178

The official training folder has more than sixteen times as many happy images as disgust images. A model could therefore improve its overall score by favouring common expressions while making little progress on disgust. We needed to inspect each class as well as the average.

2.2 One preparation rule for training and real photographs

Every face passes through the same preparation function: convert to grayscale, resize to 48×48 , and divide pixel values by 255. The resulting input has shape $48 \times 48 \times 1$ and values between zero and one. Normalisation controls the numerical scale presented to the network; it does not remove the class imbalance.

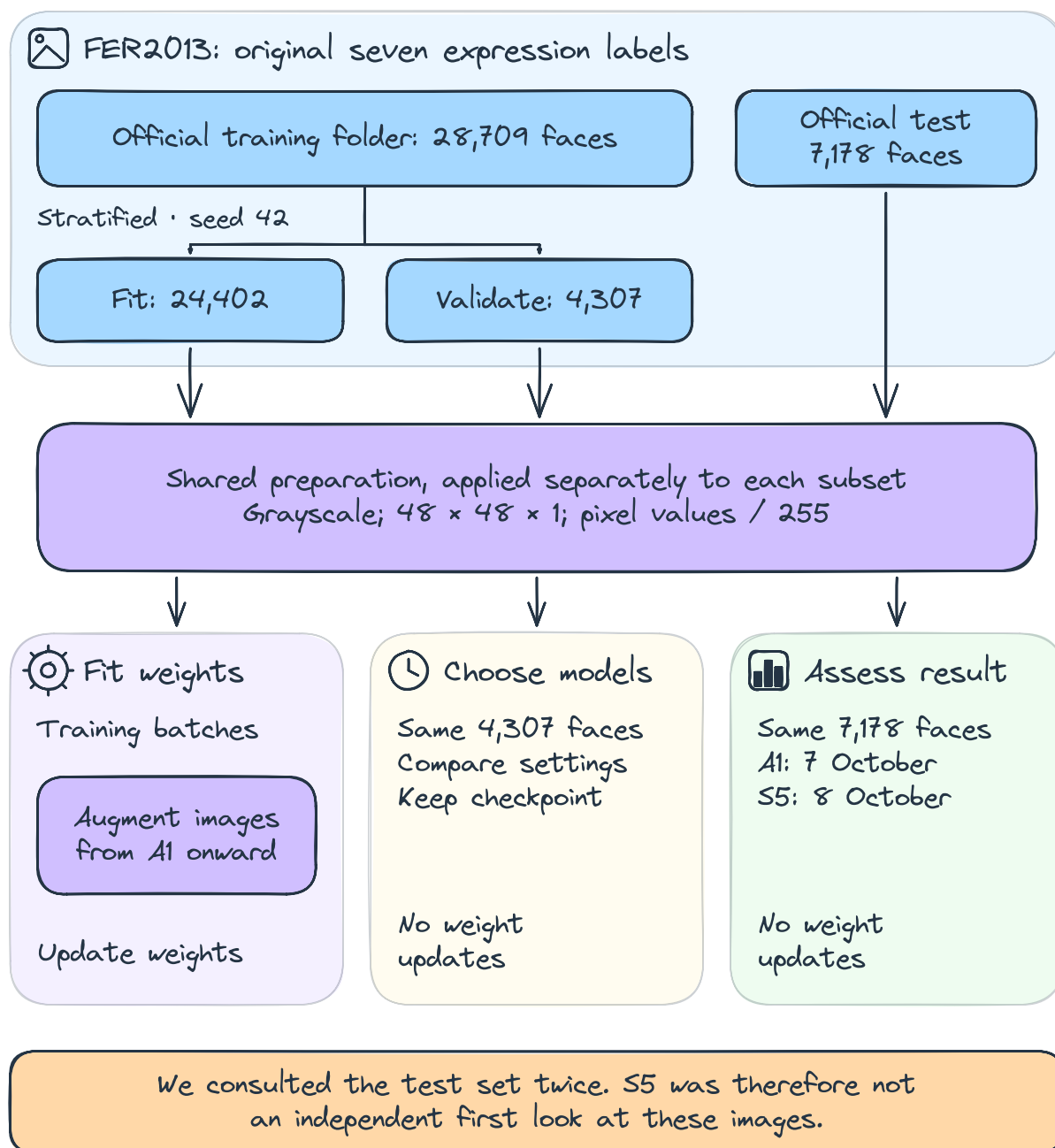
We encode the seven labels as integers in a fixed order. The corresponding output positions always mean the same expressions. This simple consistency matters when the classifier later receives a crop from a colour photograph. A different class order or preparation rule would change the meaning of an otherwise valid prediction.

Augmentation was added only to training batches. Validation and test images remained unchanged, so a candidate was judged on the same examples as its predecessors. Different training seeds in the later study changed the training randomness, not this data split.

Evidence: P2: data loader, recorded split sizes and dataset counts. See [Data and evaluation](#).

Give each image one role

One fixed split; three distinct uses of the data



■ Data ■ Processing ■ Selection ■ Result ■ Caution

Arrows show data routes; the subsets are never pooled together.

The same preparation serves three different jobs: learning weights, selecting a model and assessing it. Augmentation changes fitting batches only. The two test consultations remain visible because the second was informed by the first result. Evidence: P2 and P6.

3 What our first network could learn

The assignment asked us to build a simple dense network before moving to a CNN. We called it B0. Flattening turns the 48×48 image into a list of 2,304 pixel values. A dense layer with 128 neurons combines those values, and a final layer produces seven scores. Flattening keeps the values and their fixed positions in the list; it no longer represents neighbourhoods explicitly as a two-dimensional image.

A weight controls how strongly an input contributes to a neuron's calculation. A bias shifts that calculation. ReLU keeps positive values and replaces negative ones with zero, giving the network a nonlinear response. Without nonlinear activations, stacked dense layers could still be replaced by a single dense calculation.

3.1 How a labelled face changes the model

During a forward pass, the network calculates its output from the current weights. Softmax converts the seven output values into positive scores summing to one. We take the largest as the predicted class. The single sigmoid output used in binary classification would not provide a separate score for all seven expressions.

The loss penalises assigning too little probability to the correct class. We use sparse categorical cross-entropy because each target is a single integer label. Backpropagation calculates how changes to the weights would affect this loss. Adam uses those gradients to update the weights, with the learning rate controlling the scale of the updates. It repeats this process over batches of 64 images; one epoch means a pass through the training set.

Prediction is shorter. Once weights are saved, a new face passes forward through the network without a known label, a loss calculation for learning, or a weight update. The model applies what it learned; it does not learn from every photograph we show during the demonstration.

3.2 What five epochs established

B0 has 295,943 trainable parameters. Nearly all are in the first dense layer: 2,304 inputs multiplied by 128 neurons, plus 128 biases, gives 295,040 parameters. The seven-class output adds 903. After five epochs, its validation accuracy was 36.08% and its validation loss was 1.6603. We saved its final weights.

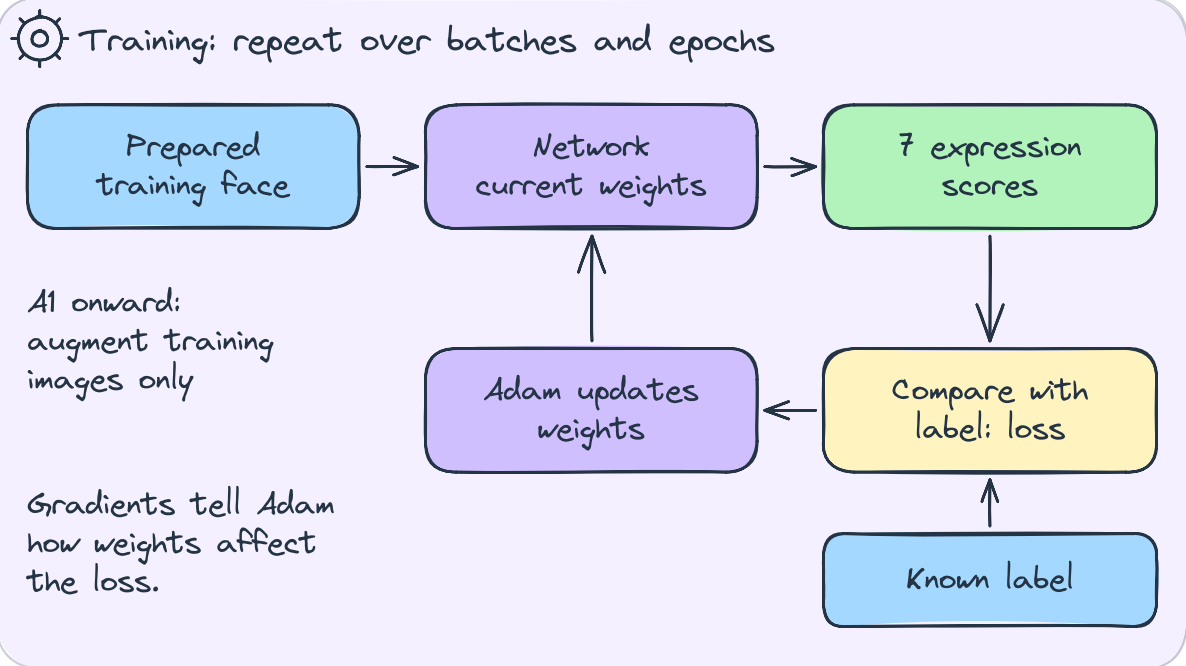
For perspective, choosing uniformly between seven labels has an expected accuracy of about 14.3%. Always predicting happy would score 24.71% on the official test distribution. These are calculated reference points, not models we trained or evaluated as part of B0. They show why a score around 57% later in the project could be disappointing to us while still being well above simple guessing.

B0's training and validation losses remained fairly close over its five epochs. That short run did not show marked overfitting. It also did not tell us how well a more carefully tuned dense network could perform. We used B0 as the requested reference and moved on to testing an architecture suited to image structure.

Evidence: P3: B0 history and model definition. Calculated guessing references use the seven labels and test counts. See [Learning versus prediction, MLP versus CNN](#).

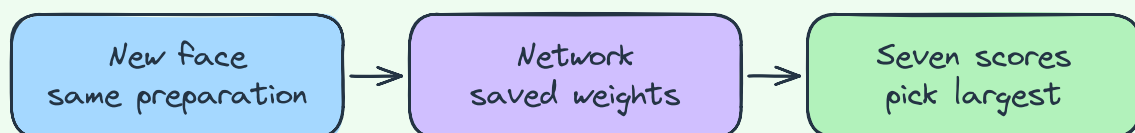
Learning and using a model

Training changes weights; prediction reuses saved weights



🕒 Validation: measure, then choose a checkpoint
Use prepared faces with current weights to measure loss.
No augmentation and no weight update.

📁 Prediction: use the selected, saved weights



No known label, learning-loss loop or weight update.

■ Input ■ Processing ■ Comparison ■ Output

Arrows show flow; feedback returns updated weights.

Learning changes the weights through repeated comparisons with labels. Validation measures the current model, and prediction uses saved weights. Augmentation was introduced with A1; it was absent from the first baseline. Evidence: P2 to P4.

4 Why we tried convolutions

A small filter can examine neighbouring pixels and reuse the same weights across the image. This gives a CNN a useful starting assumption: a local visual pattern may matter in more than one position. We did not have to tell it which exact patterns represented an expression; the filter weights were learned from labels.

C0 uses three blocks, each containing a convolution, ReLU and max pooling. A 3×3 kernel looks at a small neighbourhood; stride one moves it one pixel at a time. Same padding preserves the image width and height through the convolution. Each filter produces a feature map. Pooling then keeps the largest response in each 2×2 region, halving the spatial dimensions.

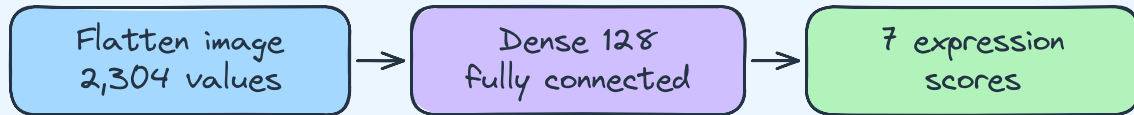
C0 stage	Output shape	Parameters
Input	$48 \times 48 \times 1$	0
Convolution, 32 filters	$48 \times 48 \times 32$	320
Max pooling	$24 \times 24 \times 32$	0
Convolution, 64 filters	$24 \times 24 \times 64$	18,496
Max pooling	$12 \times 12 \times 64$	0
Convolution, 128 filters	$12 \times 12 \times 128$	73,856
Max pooling, then flatten	$6 \times 6 \times 128$; 4,608	0
Dense, ReLU	128	589,952
Dense, softmax	7	903
Total		683,527

The dense layer still accounts for most parameters. That gave us a concrete reason to test a smaller dense layer later. More filters produce more feature maps, but also more computation and parameters; we started with 32, 64 and 128.

From a list of pixels to local patterns

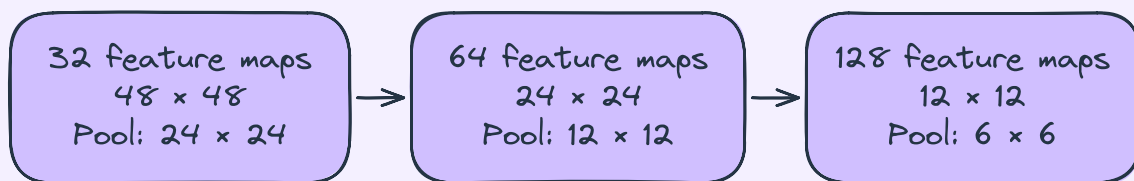
The same 48×48 grayscale face enters both models

 B0: a dense network



295,943 parameters · final weights after 5 epochs

 C0: convolution, ReLU and pooling in three blocks

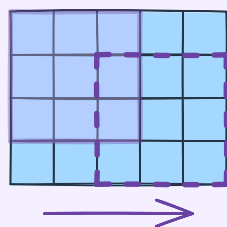


Flatten $6 \times 6 \times 128 = 4,608$; Dense 128; 7 scores

683,527 parameters · each map comes from one learned filter

 Two operations that use image structure

Local filtering



One 3×3 filter moves over pixels. Its weights are shared at every position.

2×2 max pooling



Keep the largest response.

B0 ran 5 epochs; C0 retained epoch 7 and stopped at 12 (maximum 30). The gain cannot be credited to architecture alone.

 Data  Processing  Output  Caution

Arrows show flow or movement; dashed square = next filter position.

Convolution reuses the same local filter at different positions. Pooling reduces each map while retaining its strongest local responses. These are schematic operations, not identified emotion detectors; the training budgets also differed. Evidence: P3.

4.1 The best model was not the last epoch

We allowed up to 30 epochs and monitored validation loss. Early stopping waited five epochs without improvement and restored the best weights. C0's best checkpoint was epoch seven: 55.65% validation accuracy and loss 1.2320. Training continued to epoch twelve, where validation loss had risen to 1.8986 while training loss had kept falling. The model was becoming better at fitting its training examples without improving on the held-out validation faces.

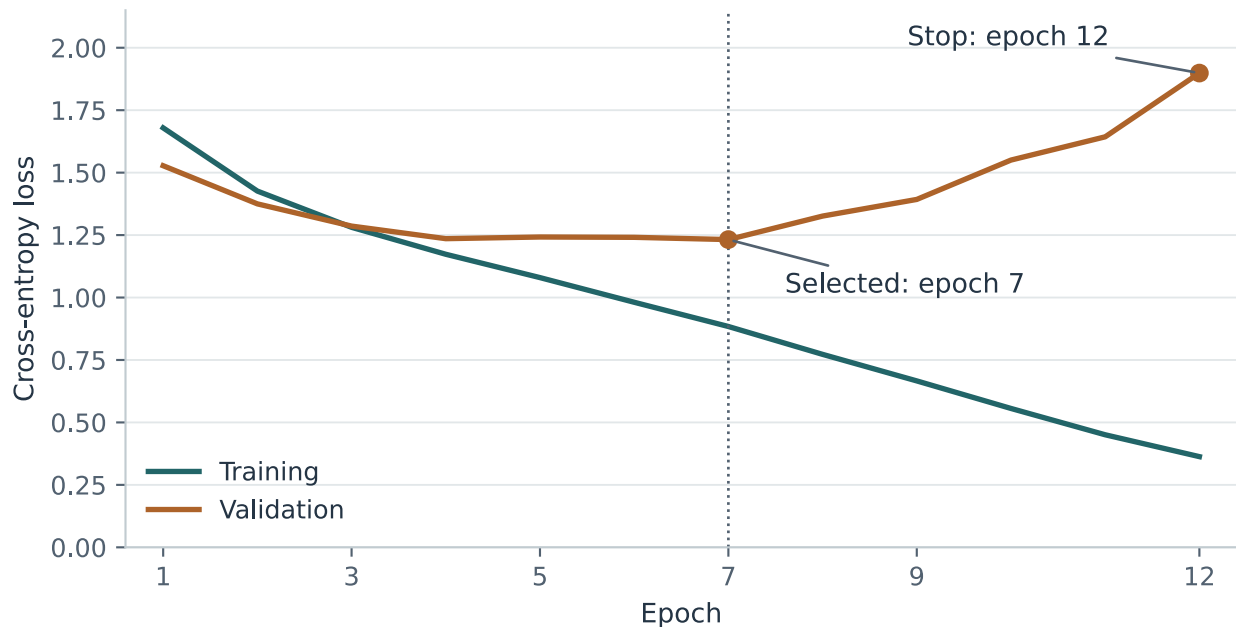


Figure 2: C0's saved learning curves. We retained epoch 7 and stopped at epoch 12, before the planned maximum of 30. Only the recorded epochs are shown. Source: P3, C0_history.json.

The gain over B0 was 19.57 percentage points of validation accuracy. That supported continuing with CNNs in our project. It was not a controlled measurement of architecture alone: B0 ran for five epochs and kept its final weights, while C0 had a larger budget and selected its best checkpoint.

The next question came from C0's behaviour after epoch seven. We wanted to find out whether reducing capacity, adding regularisation or taking smaller updates would improve its performance on unseen examples.

Evidence: P3: C0 saved history, checkpoint metrics and architecture. See MLP versus CNN, Checkpoints and stopping.

5 Which expressions were we missing?

Accuracy gives every image equal weight. That is easy to interpret, but with our class distribution it gives happy far more influence than disgust. C0's overall validation accuracy did not tell us whether it had learned all seven classes. We therefore inspected the confusion matrix, class reports and individual errors before deciding what to try next.

A confusion matrix counts where each true class ends up. A row for fear can show how often fear was recognised and how often it was called sad, angry or surprise. It lets us ask a more useful question than simply whether a model has improved: which mistakes changed?

5.1 Three measures answered different questions

Measure	How we used it
Accuracy	The fraction of images given the correct label. Useful for an overall count, but influenced by frequent classes.
Cross-entropy loss	The penalty for assigning too little probability to the correct label. A confident wrong answer can be costly. We used validation loss to select checkpoints and candidates.
Macro F1	Calculate F1 separately for each expression, then average the seven scores equally. This gives rare classes the same weight as common ones.

For one class, precision asks how often its predictions are correct. Recall asks how many of its actual examples were found. F1 combines those two measures through their harmonic mean. A model that almost never predicts disgust can avoid some false alarms yet still have poor recall for disgust. Macro F1 makes that failure visible alongside the performance on happy.

C0's validation macro F1 was 0.512, compared with 0.244 for B0. The CNN's improvement therefore extended beyond the overall accuracy. It still left a cluster of difficult distinctions involving sad, neutral and fear. Later models continued to confuse these expressions, even as their total number of correct predictions increased.

5.2 An error image is evidence, but its cause needs care

Some confident mistakes looked ambiguous to us: smiling faces labelled sad, fear or neutral, and open mouths assigned different expression labels. We also encountered a warning pictogram labelled neutral and a stock-image watermark. The pictogram is a concrete problem with the input collection. A smile on a face labelled sad is less conclusive: our visual impression is not an independent annotation study.

We considered several possible causes, including limited resolution, scarce examples, shared facial cues and annotation noise. We did not isolate those causes experimentally. A high softmax score did not settle the question either. It describes the model's preference among seven labels, and we had not calibrated those scores against their empirical correctness.

Evidence: P3, P4 and P6: notebook error analyses and saved metrics. See [Competing metrics](#).

6 Testing our explanations separately

We wanted to interpret the next results, so we changed one main factor at a time. E1 reduced the dense layer from 128 to 64 neurons. E2 kept the original size and added Dropout at a rate of 0.3 after the dense layer. E3 kept the original architecture and halved Adam’s learning rate from 0.001 to 0.0005.

These were three branches from C0. E3 did not inherit E2’s Dropout, and E2 did not inherit E1’s smaller layer. The split, seed 42, batch size 64 and maximum budget of 30 epochs were held fixed, with patience five and restoration of the best validation-loss checkpoint.

Run	Main change	Val. accuracy	Val. loss	Parameters
C0	Reference CNN	55.65%	1.2320	683,527
E1	Dense 64	54.24%	1.2283	388,103
E2	Dense Dropout 0.3	55.72%	1.1956	683,527
E3	Learning rate 0.0005	55.93%	1.1924	683,527

6.1 A smaller model made a useful tradeoff visible

E1 removed 43.22% of C0’s parameters. Its validation loss improved very slightly, but accuracy fell by 1.42 percentage points. We did not select it for the next stage. We could remove much of the parameter count without a proportionate drop in performance, although model size was not our selection criterion.

6.2 Regularisation and smaller updates both helped loss

Dropout randomly suppresses some activations during training. The intention is to reduce reliance on particular internal connections; it is inactive during normal prediction. E2 lowered validation loss to 1.1956, although overfitting remained visible. E3 lowered it slightly further, to 1.1924, by changing only the learning rate. Smaller updates were therefore worth keeping as a working configuration.

Our selection rule had been fixed before these runs: minimise validation loss, using validation accuracy if losses were exactly tied. E3 ranked first under that rule, so we used it as the provisional reference for augmentation.

The gap between E2 and E3 was only about 0.0032 loss. With one training seed, we could not tell how stable that ordering would be under different initial weights and batch randomness. We treated the winner as a practical next step. This small gap later became one reason to repeat promising comparisons across seeds.

We also kept accuracy and loss from the same saved checkpoint. Taking the lowest loss from one epoch and the highest accuracy from another would describe a model that we had never actually saved.

Evidence: P4: experiments.csv and E1/E2/E3 histories. See [Experiment history](#), [Competing metrics](#).

7 What augmentation improved

Instead of showing exactly the same training image on every pass, we added horizontal flips, rotations up to 18 degrees, translations and zoom up to 15%, and contrast changes up to 20%. These produce alternative views of existing examples. They do not create independent people or repair questionable labels.

We built A1 from E3's configuration with fresh weights. Continuing from E3's checkpoint would have mixed augmentation with extra training on already learned weights. Keeping the architecture, split and original training budget made the comparison easier to interpret.

The transformations use Keras layers called from the training data pipeline, outside the saved classifier. This preserved E3's classifier architecture and let us test the transformations separately. Validation and test batches never received these random changes.

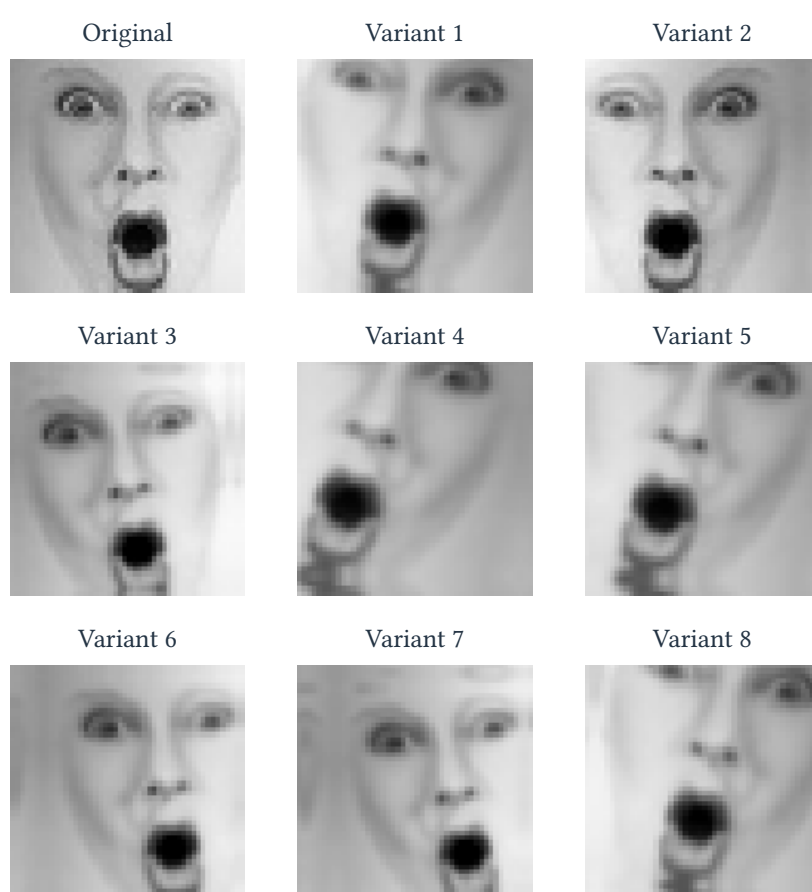


Figure 3: One fear-labelled fitting image and eight saved A1 augmentation samples, read left to right and top to bottom. We have rearranged the notebook's horizontal strip for readability. The transformations change the training view while retaining its original label; validation and test images receive no random augmentation. Source: supplied notebook, augmentation output; P4 and P9.

Run	Val. accuracy	Val. loss	Macro F1	Best / run epochs
E3	55.93%	1.1924	0.507	7 / 12
A1	57.72%	1.1263	0.502	24 / 29

A1 improved validation accuracy by 1.79 percentage points and delayed the best checkpoint from epoch seven to epoch 24. Early stopping ended training at epoch 29, just before the maximum of 30. It did not stop because it reached the cap. Validation loss rose from 1.1263 at the selected checkpoint to 1.1566 at the end.

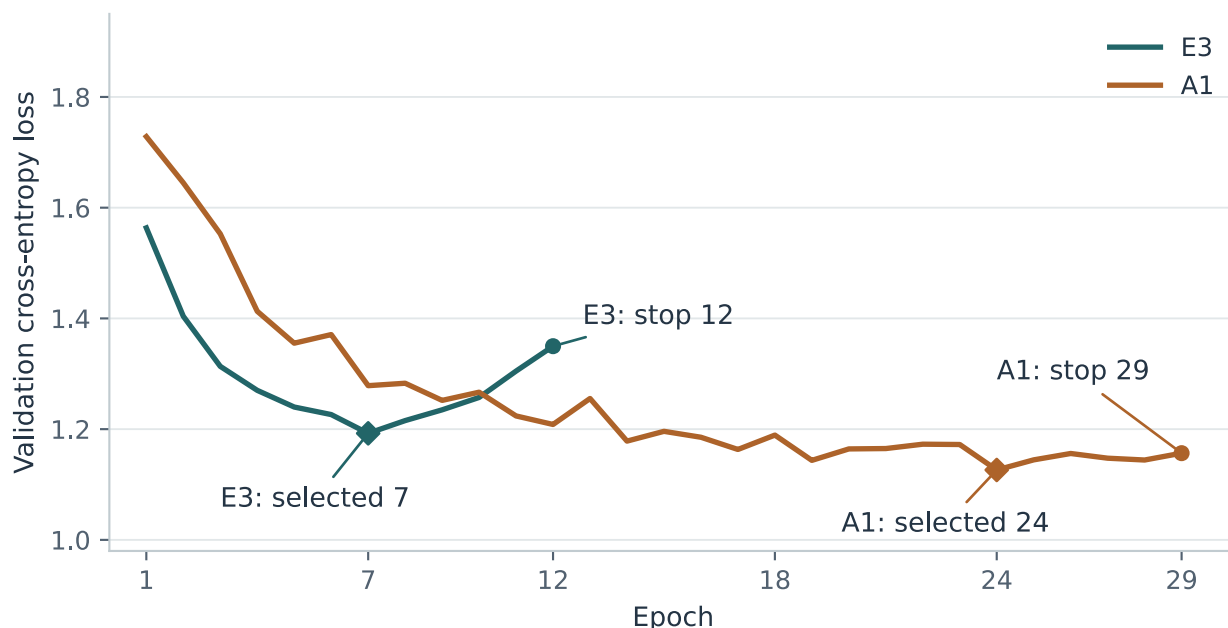


Figure 4: E3 and A1 on the same validation split, with one training seed each. Both had a maximum of 30 epochs. A1's best checkpoint came later and had lower validation loss. We show validation only, because A1's training measurements use augmented images. Source: P4, E3_history.json and A1_history.json.

The macro F1 moved in the other direction. In particular, disgust F1 fell from about 0.253 to 0.182. We suspected that some transformations could obscure small expression cues, but did not test that explanation separately. The reported training accuracy also used augmented images, while validation used unchanged ones. Their small gap could not prove either an absence of overfitting or a lack of model capacity.

7.1 The first test and a first complete photo system

A1 had the lowest validation loss, so we selected it and evaluated it on the official test on 7 October. Accuracy was 57.55%, loss 1.1317 and macro F1 0.508. Happy was much easier than disgust or fear. The small validation-to-test gap was encouraging for this run, while the class report still showed substantial errors.

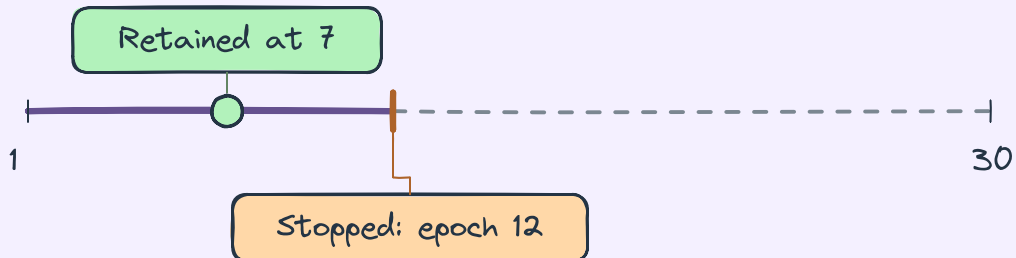
We then connected A1 to YuNet, a pretrained face detector available through OpenCV. YuNet found the face regions; our classifier assigned an expression to each prepared crop. Three Apollo crew photographs each produced three face boxes. That gave us a first working system for a whole photograph, though these few front-facing portraits could not establish its reliability on harder images.

Evidence: P4 and P6 for A1; P7 for the first photo pipeline. See Checkpoints and stopping, Competing metrics, Whole-photo processing.

Saving is not the same as stopping

Epoch timelines; see the measured CO and E3/A1 loss curves.

🕒 CO · initial CNN



🕒 A1 · augmented training



📁 Use the retained weights

Both runs stopped after five epochs without a new lowest validation loss. Neither used its full 30-epoch budget.

CO and A1 also use different learning rates; this pair does not isolate augmentation. E3 versus A1 is the controlled comparison.

■ Saved checkpoint ■ Training stop

Solid: epochs run. Dashed: unused budget. Maximum: 30 epochs.

The saved checkpoint, stopping epoch and maximum budget are different events. Dashed timeline segments are unused budget, not predicted losses. The measured curves appear in Figure 2 and Figure 4. CO and A1 also differ in learning rate; E3/A1 is the single-seed augmentation comparison. Evidence: P3 and P4.

8 A more careful search

A1 was useful, but its 57.55% test accuracy still left many faces incorrectly labelled. We had read a human-accuracy estimate of $65 \pm 5\%$ in Goodfellow et al. and a 73.28% result for a tuned VGG in Khairuddin and Chen. Those gave us reasons to keep investigating, rather than a score we could expect to reproduce directly. The VGG study used a different partition protocol, much longer training and ten-crop averaging at prediction time; its final tuning also used validation data for training. Our 7,178-image held-out set and locally created validation set were not a matched reproduction.

Our own results raised more immediate questions. A1's best checkpoint came late in the original budget. Fear and disgust remained weak. E2 and E3 had almost tied under one seed. We suspected that network capacity, training time and randomness were worth examining, without treating any one of them as an established cause.

On our M3 Max Mac, the smaller CNNs took roughly ten seconds per epoch in the local runs, with no GPU exposed to TensorFlow. That made an overnight study feasible. A wider 64/128/256-filter network had taken about 92 seconds per epoch in our recorded timing, so we retained 32/64/128 filters. The 28 completed research runs accumulated about 6.6 hours of recorded runtime.

8.1 A ladder that kept only accepted changes

We first reran A1 locally as R0, then tested each proposal on seeds 42, 43 and 44. The next proposal inherited the last accepted configuration. A rejected option did not silently remain in later models. Selection used the lowest mean validation loss across the three seeds.

Run	Change from accepted reference	Mean val. loss	Decision
R0	A1 rerun locally	1.1319	Reference
R1	BatchNorm after convolutions	1.2218	Reject
R2	Two convolutions per block	1.0523	Keep
R3	Dropout in blocks and dense layer	1.0825	Reject
R4	60 epochs, adaptive learning rate, patience 8	1.0299	Keep
R5	Class weighting	1.1266	Reject

R2 therefore had no BatchNorm, and R4 did not inherit R3's Dropout. R4 changed a training policy comprising the budget, stopping patience and learning-rate reduction. Its result cannot isolate the contribution of each of those settings. Three seeds gave us a view of training variability on this split, not a measure of uncertainty over every possible dataset or population.

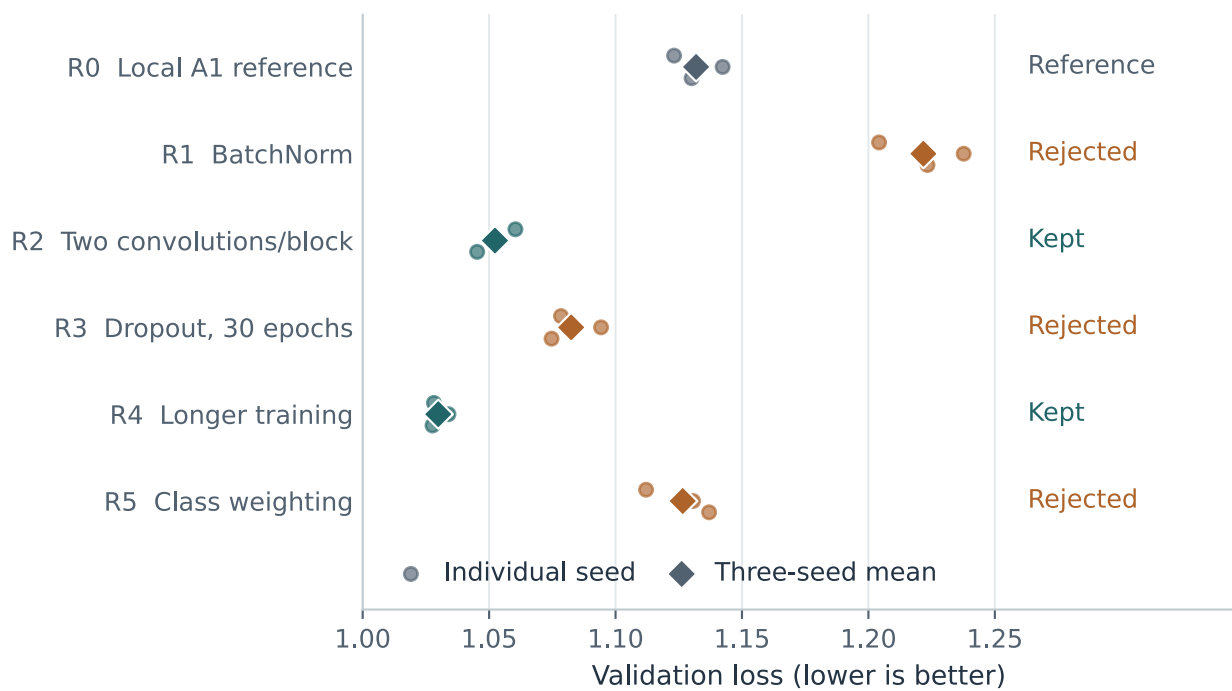
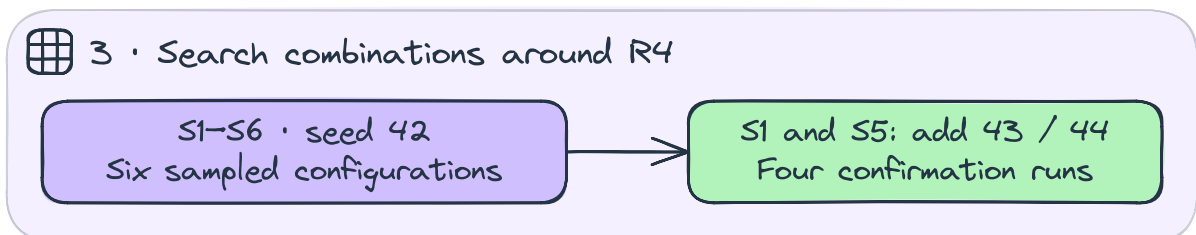
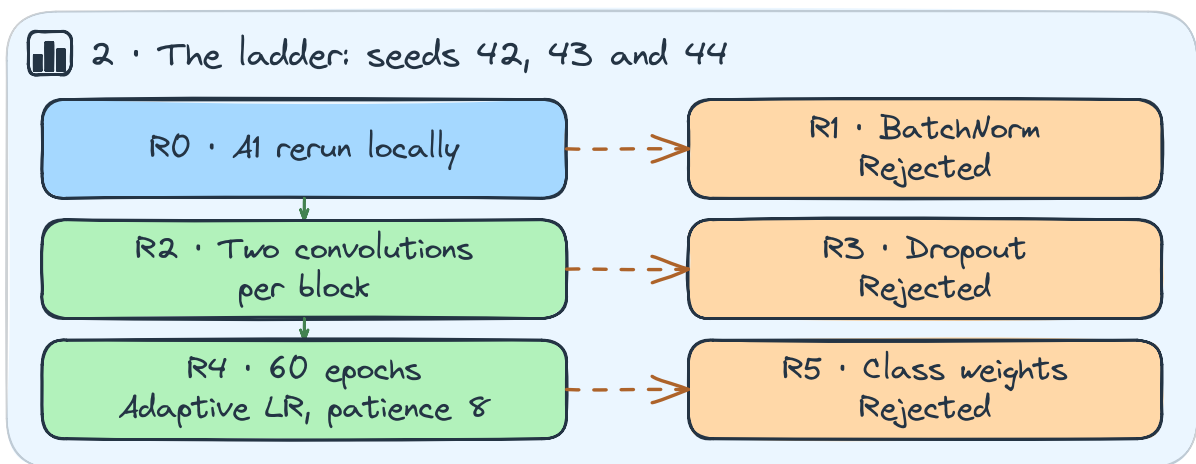
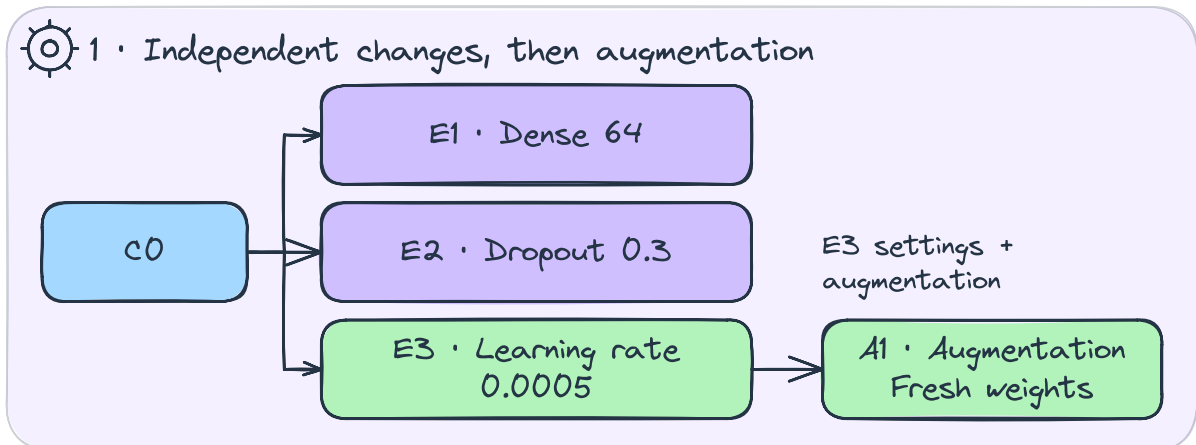


Figure 5: Each small point is one seed; each large marker is the three-seed mean. Lower validation loss was our selection criterion. These are separate configurations: rejected changes did not carry forward. All seeds used the same split, so their spread is not a confidence interval for other datasets. Source: P5, R0 to R5 records and `ladder.json`.

Evidence: P5 and our timing recollections; literature context R3 and R4. See [Experiment history](#).

Our experiment history

Arrows trace configurations, not continued training from saved weights.



28 runs = 18 ladder + 6 search + 4 confirmations

12 research configurations: R0-R5 and S1-S6

■ Reference
 ■ Trial
 ■ Retained
 ■ Rejected

Solid: configuration ancestry. Dashed: rejected ladder trial.

Arrows trace configuration choices, not resumed model weights. Only accepted ladder changes carry forward. Dropout reappears in S5 alongside other changed settings, so the history does not isolate an interaction. The research phase contains 28 runs across 12 configurations. Evidence: P4 and P5.

9 Rejected settings still taught us something

BatchNorm surprised us. It normalises intermediate activations using batch statistics during training and running statistics during prediction. We had expected it to help training, yet R1’s mean validation loss rose to 1.2218 from R0’s 1.1319. The best checkpoints occurred at epochs 12 to 18; stopping actually occurred at epochs 17 to 23. A mismatch between augmented training batches and clean validation images was one possible explanation, but we did not test it.

R2’s extra convolution in each block lowered mean validation loss to 1.0523. Each block could now perform two learned local transformations before pooling reduced its resolution. This was the first accepted architectural change in the ladder.

R3 added Dropout of 0.25 after each block and 0.5 near the output. All three runs used their 30 epochs and selected epoch 29. Their mean loss was worse than R2’s, so we rejected this setting under that budget. The late best epochs suggested that longer training might be worth trying. They did not prove that more time would reverse the result.

R4 extended training to 60 epochs and reduced the learning rate when validation loss plateaued. Its mean loss improved to 1.0299. Class weighting in R5 then increased the influence of scarce labels during training. It slightly improved mean disgust F1, from 0.350 to 0.366, but fear fell from 0.467 to 0.410 and macro F1 from 0.569 to 0.537. We rejected R5 on the overall criterion while retaining that small class-specific gain in our interpretation.

9.1 Searching combinations around R4

We sampled six combinations of learning rate, batch size, Dropout and dense-layer size. Each first ran with seed 42. The two best, S1 and S5, received seeds 43 and 44 as well. This gives 18 ladder runs, six initial search runs and four confirmations: 28 runs across 12 configurations.

One candidate, S4, predicted happy for every validation image, scoring 25.12%, the happy share of that set. Its relatively high initial learning rate was a suspect, not an isolated explanation. The failed run showed that adding capacity and tuning settings could also produce a useless classifier.

S5 won mean validation loss. It uses two 3×3 convolutions per block, 32/64/128 filters, Dropout 0.25 after pooling, Dense 256 and Dropout 0.3 before the output. Its initial learning rate is approximately 0.000655, batch size 32, and it retains R4’s adaptive reduction and 60-epoch budget. It has 1,468,135 parameters, mostly in the dense layer. Dropout’s presence in S5 made us cautious about rejecting it in every setting, but several settings changed together, so no single interaction was demonstrated.

Evidence: P5: all run records, ladder decisions and research summary. See [Experiment history](#).

10 Why S5 won, and what that means

S5 had the lowest mean validation loss among the candidates confirmed over three seeds. R4 remained a close alternative, with fewer parameters and a better mean macro F1. We chose S5 using the rule we had set for the study, while keeping that tradeoff visible.

Three-seed comparison	R4	S5
Mean validation loss	1.0299	1.0232
Loss standard deviation	0.0035	0.0121
Mean validation accuracy	62.22%	62.84%
Accuracy standard deviation	0.88 points	0.54 points
Mean macro F1	0.569	0.562
Parameters	877,287	1,468,135

The loss gap, about 0.0067, is smaller than S5's variation across seeds. These standard deviations describe three training runs; they are not confidence intervals or evidence of statistical superiority. We also reused one validation set for many decisions, which limits how broadly we can interpret its ordering.

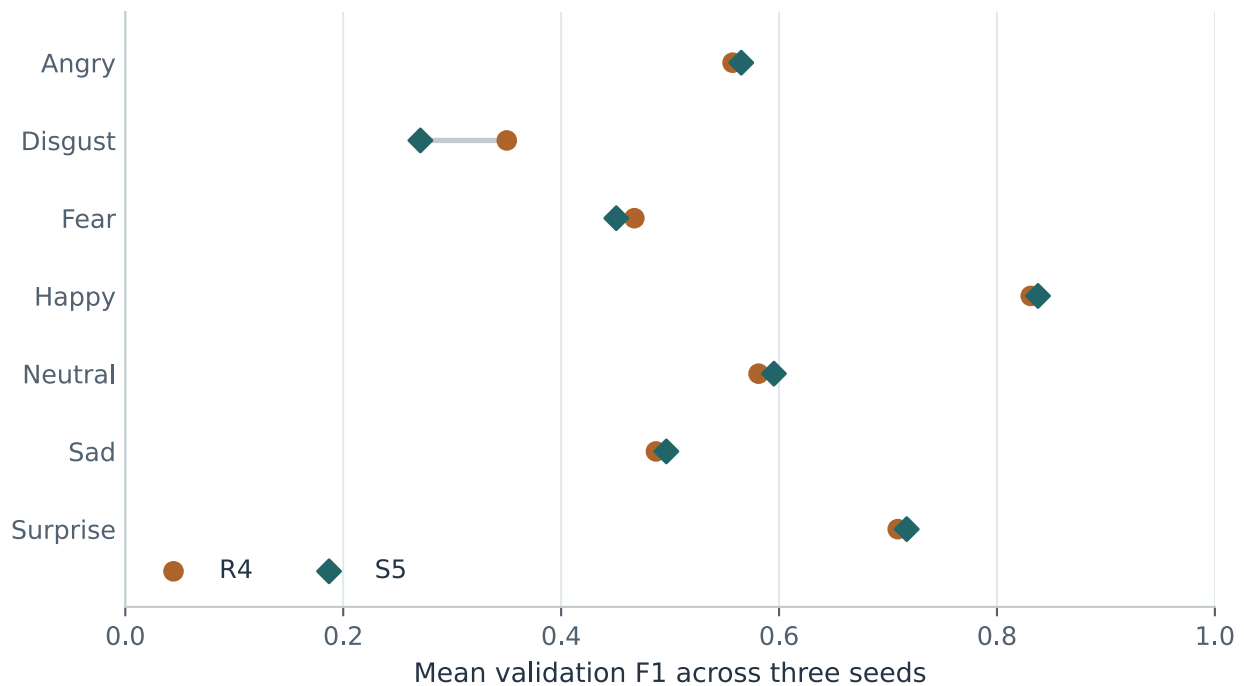


Figure 6: Mean validation F1 for each expression across three seeds. Each point averages scores from separately trained models; it is neither an ensemble score nor a test result. The class differences help explain R4's higher macro F1, while S5 won on mean validation loss. Source: P5, R4 and S5 records, `val_f1` fields.

Different scores, different strengths

All values here describe validation, not the official test set.



Accuracy

Correct predictions as a share of all images



Loss

Penalty from the score assigned to the correct label



Macro F1

Each class has an equal share in the mean F1



One run per model · seed 42

Metric	E3	A1
Loss (lower wins)	1.192440	1.126252
Accuracy	55.93%	57.72%
Macro F1	0.507	0.502
Disgust F1	0.253	0.182
Parameters	683,527	683,527



Three-seed means · seeds 42, 43 and 44

Metric	R4	S5
Loss (lower wins)	1.0299	1.0232
Accuracy	62.22%	62.84%
Macro F1	0.569	0.562
Disgust F1	0.350	0.271
Parameters	877,287	1,468,135

We selected the lowest mean validation loss. S5 improved that criterion, while R4 kept higher macro F1 with fewer parameters.

■ Measurements ■ Scoring ■ Equal class weight

Green cell: lower loss. Orange cell: weaker F1 in this comparison.

A lower loss can coexist with weaker results for some expressions. Our declared loss criterion selected S5, while R4 retained better mean macro F1 with fewer parameters. Keep the single-run E3/A1 comparison separate from the three-seed R4/S5 comparison. Evidence: P4 and P5.

10.1 A second, explicitly recorded test consultation

The S5 checkpoint we use in the demonstration comes from seed 42. It retained epoch 58 after 60 epochs, with validation accuracy 63.27% and loss 1.0096. Its test accuracy on 8 October was 62.44%, loss 1.0047 and macro F1 0.562. The comparison below uses the saved test predictions of A1 and this S5 checkpoint, not the average of three S5 models.

Test F1 by class	A1	S5
Angry	0.498	0.545
Disgust	0.252	0.308
Fear	0.324	0.406
Happy	0.802	0.852
Neutral	0.528	0.588
Sad	0.456	0.487
Surprise	0.694	0.749

S5 improved accuracy by 4.89 percentage points over A1 and improved F1 for all seven expressions on this test. Disgust remained the hardest class. Sad was called neutral 292 times, and fear was called sad 213 times. More correct labels did not remove the familiar confusions.

The validation-to-test accuracy gaps were small for A1 and the selected S5 run. That observation does not quantify selection bias. We already knew A1's test result when we continued the project, even though the later choices used validation metrics. The second test result therefore deserves less confidence as a completely unseen final check.

Evidence: P5 and P6: seed-level validation records and both saved test evaluations. See [Competing metrics](#).

11 Three photographs were not enough

A classifier trained on cropped faces cannot locate those faces in a whole photograph. We needed a detector to propose their rectangles first. We began with YuNet because it was small, pretrained for faces and available through OpenCV. We later integrated a pretrained YOLO11n-face model. We did not train either detector. A standard COCO person detector would provide body boxes, which would not directly supply the facial crops our classifier expects.

Once S5 was selected, we used it behind both detectors. On three Apollo crew photographs, both found all nine inspected faces and produced the same expression labels. YuNet was faster. Those posed, largely frontal portraits gave us too little variety to understand the tradeoff.



Figure 7: Apollo 11: YOLO finds three faces, which S5 labels neutral, happy and happy. The displayed expression scores are 0.79, 1.00 and 0.96; the separate face scores are about 0.85 to 0.87. These are recorded predictions, not measured expression accuracy. Photograph: NASA, public domain; saved Colab output, P9.

We expanded the comparison to 80 public-domain NASA photographs: 50 official ISS portraits, 20 images in orbit and ten Artemis images. At the default thresholds, we paired overlapping boxes between detectors and inspected their differences. The matching threshold was intersection over union, or IoU, of at least 0.5. IoU divides the overlapping area by the area covered by either rectangle.

Recorded comparison	YuNet	YOLO11n-face
Default score threshold	0.9	0.5
Detected boxes	359	401
Boxes unique to detector	0	42
Photos with no detected face	3	0
Mean local CPU time, including S5	35 to 37 ms	50 to 51 ms
Weight file size	232 KB	About 10 MB

All 359 YuNet detections had a YOLO match. Of YOLO's 42 additional boxes, 41 contained real faces on inspection; the remaining box mostly showed a hand. The extra faces included profiles, rotated faces and faces behind glasses or masks. In those local CPU measurements, we accepted roughly 14 additional milliseconds per photograph for this coverage and made YOLO the default. YuNet remained available.

The rotated group in Figure 8 shows why the larger collection changed our view. Both panels use S5 after detection, but the detectors supply different crops at their default thresholds. The additional boxes do not tell us whether their expression labels are correct.

11.1 The threshold explained part of the difference

Lowering YuNet's threshold to 0.7 produced 395 boxes. At 0.5 it produced 435, but inspection of its 40 boxes unmatched by YOLO found about 17 clear non-faces, 12 faces and 11 uncertain regions. Counting more boxes was not sufficient to call a detector better.

We had no reference annotations for every face in these photographs. The counts and inspection therefore support our practical choice without measuring precision, recall or mean average precision. Among the 359 common detections, 336 received the same expression with either crop. That measures agreement, not expression accuracy. Likewise, the 303 happy predictions among YuNet's 359 crops describe our outputs on this sample, not verified emotional states.

YuNet · threshold 0.9 · 2 boxes



YOLO11n-face · threshold 0.5 · 8 boxes



Figure 8: On this rotated group portrait, YuNet returns two boxes and YOLO eight. The thresholds differ, and this single example is not a precision or recall measurement. S5 supplies the expression labels in both panels. Photograph: NASA, 8 November 2013, public domain; saved Colab comparison, P7 and P9.

Why we changed the face detector

We broadened the photographs before making our choice.



A small first comparison

3 Apollo photos
9 faces found by both

Same expression labels.
Too little variety to decide.



Expand to 80 NASA photographs

50 ISS portraits + 20 in orbit + 10 Artemis images

359 matched boxes
YuNet (0.9): 359
YOLO (0.5): 401

42 extra YOLO boxes
41 inspected faces
1 hand

Box matching: IoU 0.5 or more

⌚ Local CPU + S5: YuNet 35–37 ms; YOLO 50–51 ms



Lowering YuNet's threshold

0.9 threshold
359 boxes



0.7 threshold
395 boxes



0.5 threshold
435 boxes

At 0.5: inspect 40 YuNet boxes unmatched by YOLO

about 17 non-faces

12 faces

11 uncertain

We kept YOLO for broader coverage (about 14 ms more per photo).
No full reference annotations: counts do not measure
precision, recall or expression accuracy.

■ Sample ■ Threshold ■ Evidence / choice ■ Caution

Arrows: broaden the sample or lower the threshold

Broadening the sample exposed differences hidden by the three Apollo portraits. The local CPU timings include S5 classification, and the default detector thresholds differ. Counts and visual inspection supported a practical choice; without complete reference annotations they do not measure precision or recall. Evidence: P7.

Evidence: P7: 80-photo manifest, notebook comparison and inspection notes; R11 and R12. See Choosing a detector.

12 What happens to one group photograph

We traced one ISS photograph containing eleven people. The image was resized while preserving its proportions, then placed inside the detector's 640×640 input using padding. YOLO produced 8,400 box proposals across three spatial scales. Only 99 exceeded the score threshold of 0.5.

Many proposals surrounded almost the same face. Non-maximum suppression, or NMS, kept a strong proposal and removed sufficiently overlapping alternatives. On this image it kept ten boxes and removed 89 candidates. The retained faces had groups of eight to ten near-duplicate proposals, with median overlap between 0.97 and 0.99. The notebook's step-by-step simulation matched OpenCV's selection.

Ten detections for eleven people also exposed a failure: a sideways astronaut's face was absent. The coarsest 20×20 prediction grid produced no candidate above threshold on this image. These are observations from one photograph, not general limits on the detector's scales or a measured recall for the NASA collection.

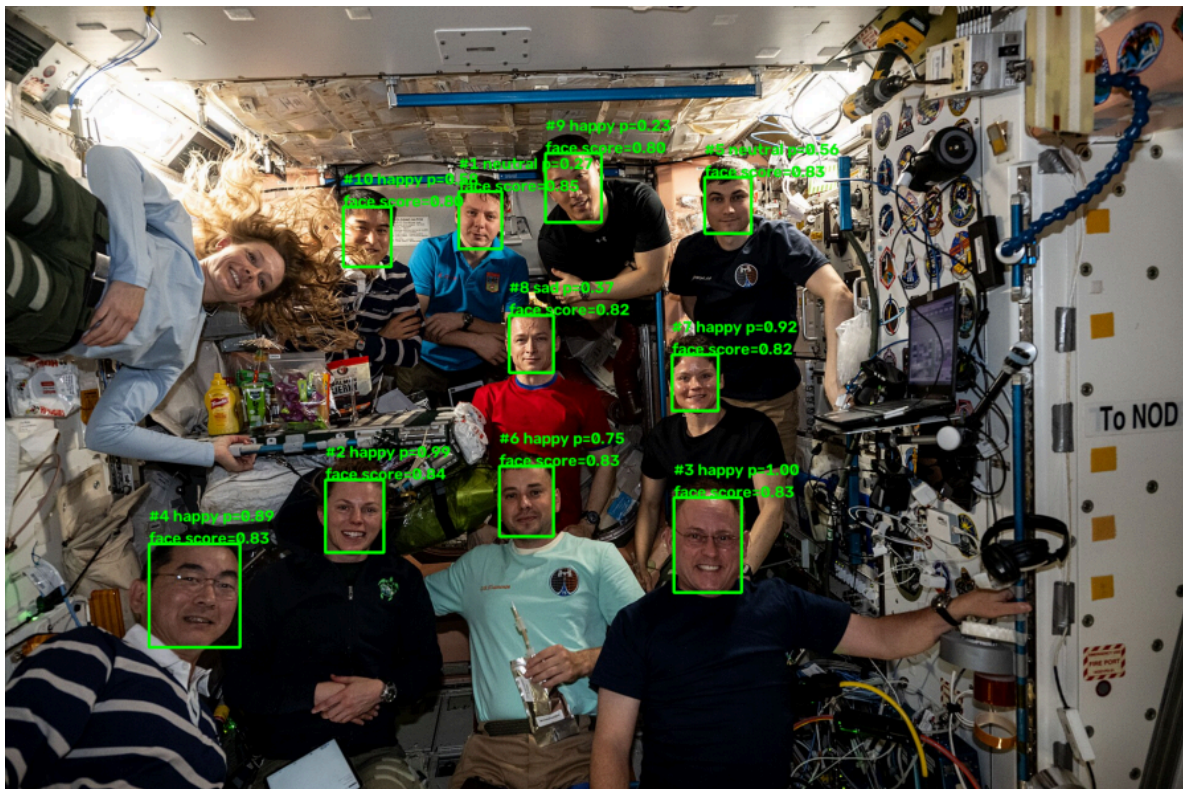


Figure 9: The recorded ISS result: ten detected faces among eleven people. The sideways face at the far left is missed. For face 9, the detector score is 0.80 while the highest expression score is only 0.23. A likely face can still leave the classifier uncertain. Photograph: NASA Johnson Space Center, 3 August 2025, public domain; saved Colab trace, P7 and P9.

12.1 The same classifier, now receiving different crops

Each retained box was mapped back to the original photograph and cropped. Our shared preparation function converted it to grayscale, resized it to 48×48 and normalised it. S5 processed the crops together and returned seven scores for each. The annotations kept the detector score separate from the expression score: finding a likely face and choosing its expression are different decisions.

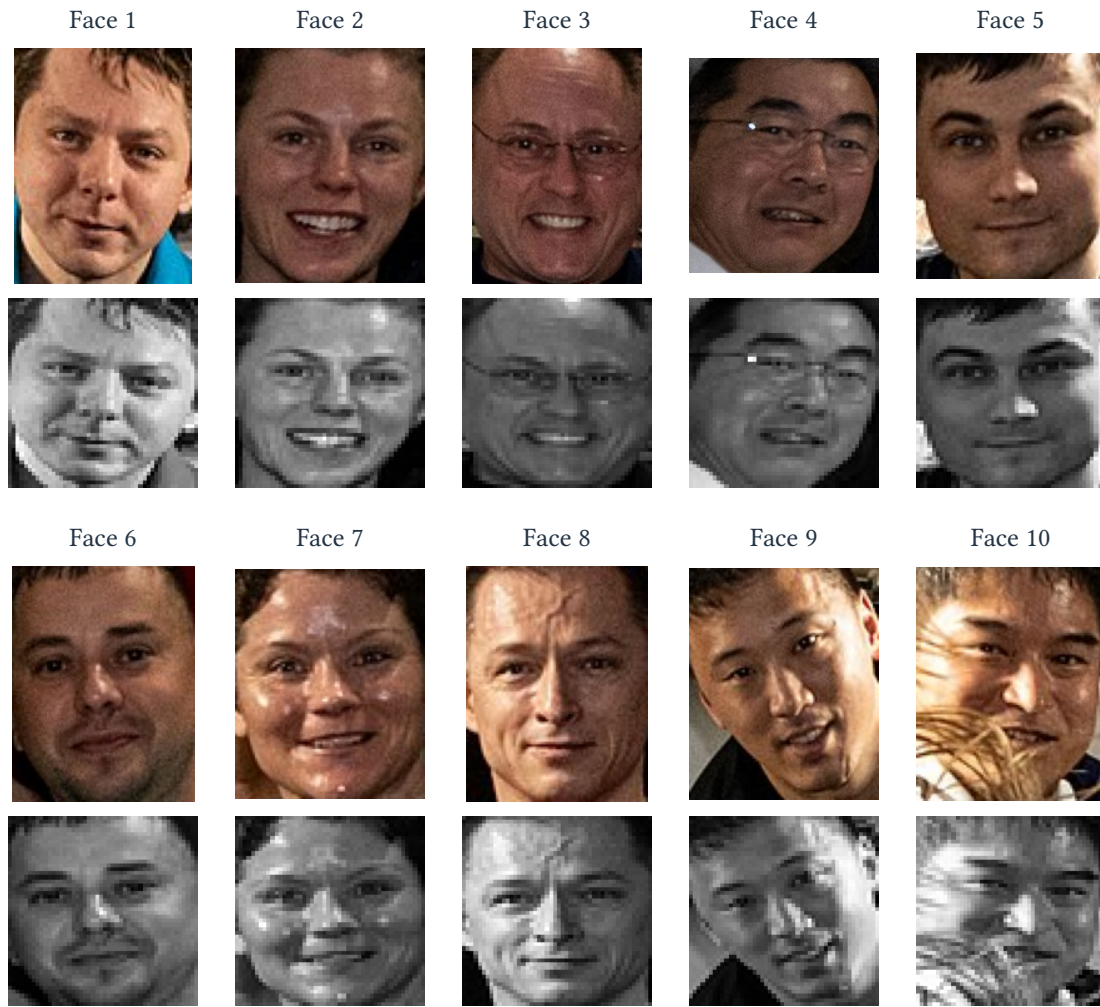
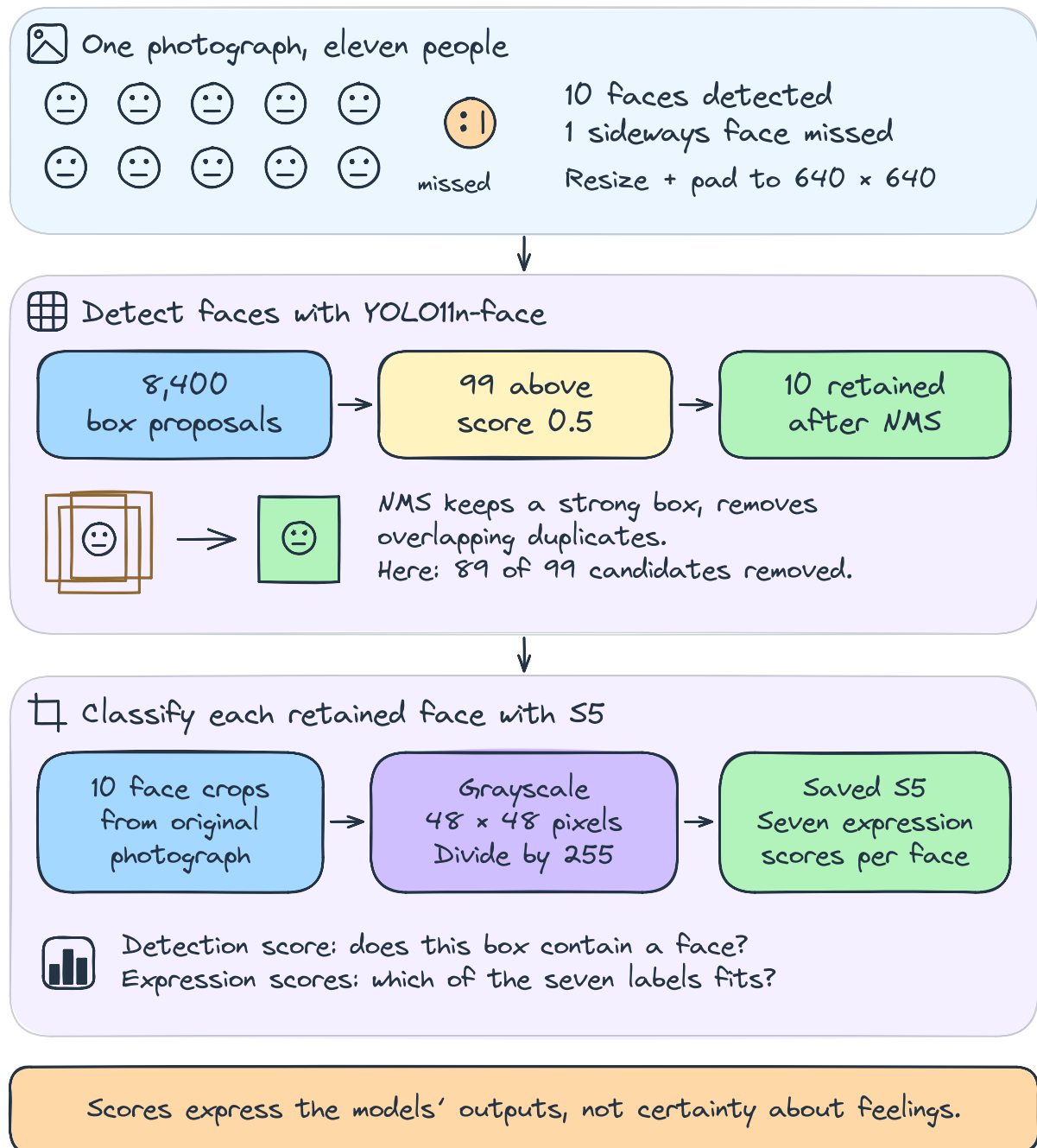


Figure 10: The same ten faces before and after preparation. In each group, colour crops are above their grayscale 48×48 inputs; the face numbers match Figure 9. The classifier receives values normalised to $[0, 1]$. These saved pairs show the loss of colour and fine detail during preparation. Original photograph: NASA Johnson Space Center, public domain; Colab trace, P7 and P9.

Some clear smiles received happy scores above 0.89. For one inclined face, no class exceeded 0.23. Limited rotation augmentation could contribute to that uncertainty, but orientation was not isolated experimentally. The intermediate feature maps helped us see changes in spatial responses; they did not prove that a particular channel represented a named facial feature or emotion.

From a photograph to expression scores

Recorded ISS example; face drawings are schematic.



 Input  Processing  Filter  Retained  Limit

Arrows: processing order

The drawing is a schematic; the counts come from the recorded ISS example. NMS removes overlapping alternatives, then S5 processes each retained face crop. One sideways face was missed. Detection and expression scores describe different stages, and neither establishes a person's feelings. Evidence: P7.

12.2 Where we stopped

We left face alignment using landmarks, FER+ labels and classifier transfer learning untested. Pretrained classifiers would have required input adaptation and another controlled comparison. Wider networks were too slow for the study we wanted to finish. We did not test budgets beyond 60 epochs, even though the selected S5 run's best checkpoint came late. We implemented photographs, not video or temporal tracking.

Preparing the demonstration also required compatibility fixes: the YOLO export needed the supported OpenCV version, and S5 was resaved for the Keras version used in Colab with unchanged weights and validation metrics. Local checks were recorded. The supplied notebook now also contains an executed Colab demonstration using YOLO and S5. Its saved outputs document that demonstration; they do not establish a fresh run of every experiment. None of these compatibility changes constitutes a new model improvement.

Our finished system can locate several faces and assign each one an expression label. Its remaining errors arise at both stages, and an apparently confident label can still be wrong. The next investigation would need to distinguish missed faces, unsuitable crops and uncertain expression labels before choosing which component to improve.

Evidence: P7 and P8: recorded ISS trace, implementation scope and compatibility history. See [Whole-photo processing](#).

Sources

Task and dataset

R1. Hanane Zerdoum. *Projet Deep Learning: reconnaissance et analyse automatique d'expressions faciales*, project assignment. Local file: `Projet_DL_Expressions_faciales (1).pdf`. Gives seven example classes, required work in parts 1 to 6 and optional extensions in parts 7 to 9.

R2. FER2013, [Kaggle distribution by msambare](#). We used its original labels. The notebook records the listing's licence as "Database: Open Database, Contents: Database Contents." FER2013 was created by Pierre Luc Carrier and Aaron Courville (R3).

R3. Goodfellow et al. (2013). [Challenges in representation learning: a report on three machine learning contests](#). Describes FER2013 and its human-accuracy estimate of $65 \pm 5\%$. Its original competition partitions differ from our local validation split.

R4. Khairuddin and Chen (2021). [Facial emotion recognition: state of the art performance on FER2013](#). Reports 73.28% with a tuned VGG. This is a published reference point under another protocol, not a directly matched experiment.

Alternatives considered

R5. Microsoft. [FER+ annotations](#). Ten annotators label each image, including eight emotions, "unknown" and "not a face." This informed an untested improvement for noisy labels.

R6. Li, Deng and Du (2017). [Reliable crowdsourcing and deep locality-preserving learning](#). RAF-DB contains 29,672 images overall; its seven-class basic-expression subset contains 15,339. These counts describe different scopes.

R7. University of Denver (2024). [AffectNet Academic Use License](#). Original agreement requiring students to request access through an academic supervisor. The current website routes acquisition through the newer AffectNet+ extension.

R8. Dimitrios Kollias. [Aff-Wild2: data and access procedure](#). Describes video annotations and a signed-agreement request. Current instructions include student access and an approximate 14-day response period; we did not measure that delay ourselves.

R9. Lee et al. (2019). [CAER project page](#). Context-aware emotion data, including the static CAER-S subset. We considered it without checking its download procedure during the project.

R10. Zhang et al. [Expression in-the-Wild, official project page](#). Provides ExpW download links. We did not investigate access or train a detector on these data.

Pre-trained detectors

R11. OpenCV Zoo. [YuNet face detector](#). Source of the pre-trained detector used in our first photograph pipeline.

R12. [akanametov/yolo-face](#) and [deepghs ONNX exports](#). Upstream face-detector project and distribution used for YOLO11n-face. We used existing weights.

Evidence index

Paths below start at `ING3_IAA_GB_DL/`, the project folder. Saved measurements take precedence over rounded narrative summaries. A hypothesis explains a possible cause; it does not establish that cause.

P1. Dataset and recollections. Notebook “Phase 1” and `src/data.py` establish the image format, class order and counts. Our notes supplied on 8 October 2026 explain the deadline, dataset choice, surprises and abandoned ideas. These notes provide our recollections, not additional experiments.

P2. Comparison protocol. `SPEC.md`, notebook “Phase 1” and `src/train.py` describe the stratified validation split, fixed seeds, checkpoint selection and untouched test input during training. The different training budgets for B0 and the CNNs limit causal comparisons.

P3. First baselines. `training/logs/B0_history.json` and `C0_history.json` contain configurations and epoch histories. Notebook “Phase 2” through “Phase 5” interprets the learning curves, confusion matrix and class-level results.

P4. Controlled changes and augmentation. In `training/logs/`, the E1, E2, E3 and A1 history files record each change. `experiments.csv` is the compact comparison. Notebook “Phase 6” and “Phase 7” explain the choices and the weaker macro F1 despite A1’s higher accuracy.

P5. Further research. `training/research/` contains individual JSON files by configuration and seed. Their saved metrics support `summary.csv`; `ladder.json` records selection decisions. These distinguish repeated runs from candidates tried once and budget exhaustion from early stopping.

P6. Final test results. `training/logs/test_evaluation.json` records A1; `test_evaluation_deepdive.json` records S5. Both store predictions, accuracy, loss and data/checkpoint hashes. They document two declared test consultations. S5’s validation average across seeds differs from the validation result of its selected checkpoint.

P7. Photographs and detection. Notebook “Phase 8”, especially the NASA comparison and “Pipeline pas à pas”, records counts, visual inspection and the NMS example. `docs/nasa_photos.csv` gives image provenance. These are recorded observations, not independently rerun for this report; without reference boxes, they establish neither detector precision nor recall.

P8. Final scope. `README.md`, `TODO.md` and `src/detect.py` describe the photograph pipeline and its defaults. `JOURNAL.md` records compatibility checks. These distinguish the implemented demonstration from untested alignment, classifier transfer learning and video work.

P9. Saved visual outputs. The six visual-result figures come from the supplied `ING3_IAA_GB_DL_PIT10T_CRAYSSAC.ipynb`, also shared through Colab. We extracted existing PNG outputs without executing cells. English labels and panel arrangements were added for this report; image content, recorded predictions and sample order were preserved. The original PNGs, pixel-crop coordinates and image hashes are stored with `docs/assets/results/manifest.json`.

Notebook SHA-256:
82b683ecce2a8f531cb0fba9069c609e2c547ee9fd3cba5070fef857340f7d37

Saved visual	Cell ID	Cell	Output
Seven classes	acf84f0b	6	0
Augmentation	1e70cded	33	0
Apollo 11	463157c7	49	2
Detector comparison	ebc1184f	59	1
ISS final result	x55-BRu0RyJD	62	16
ISS colour crops	x55-BRu0RyJD	62	11
ISS grayscale inputs	x55-BRu0RyJD	62	12

Cell and output indexes above start at zero. The Apollo scores are also saved in cell 49, output 1. The supplied Colab comparison records mean times of 319.0 ms for YuNet and 323.3 ms for YOLO, including S5. The earlier local CPU measurements in section 11 remain a separate timing record; the Colab images are not evidence for those local durations.

Photograph credits: [Apollo 11 crew](#), NASA; [Expedition 37/38 portrait](#), NASA, 8 November 2013, image [iss037e028227](#); [eleven-person ISS portrait](#), NASA Johnson Space Center, 3 August 2025, image [iss073e0420899](#). All three photographs are public domain; `docs/nasa_photos.csv` records the provenance of the two broader-sample images. FER2013 sample provenance is R2.

Parts 1 to 6: data, dense baseline, CNN, training, evaluation and comparisons. Part 7: augmentation completed; classifier transfer learning not explored. Part 8: multi-face photographs completed using pre-trained detectors. Part 9: video extension not implemented.